

Phép nối JOIN (inner, left, right, full outer)

Trong các nội dung trước, chúng ta đã làm quen với việc truy xuất dữ liệu trên một bảng đơn lẻ. Tuy nhiên, theo nguyên tắc chuẩn hóa dữ liệu (Normalization), cơ sở dữ liệu thường được phân rã thành nhiều thực thể (bảng) khác nhau để tránh dư thừa thông tin.

Ví dụ: Bảng **SinhVien** chứa thông tin cá nhân và **lop_hoc_id**, trong khi bảng **LopHoc** chứa thông tin chi tiết về tên lớp tương ứng với **id** đó. Khi có yêu cầu kết xuất một báo cáo bao gồm cả "Tên sinh viên" và "Tên lớp", chúng ta không thể chỉ truy vấn một bảng. Để liên kết thông tin giữa các thực thể này, chúng ta sử dụng phép nối **JOIN**.



1. Khái niệm về JOIN và mệnh đề ON

JOIN là một thành phần trong mệnh đề **FROM**, được sử dụng để kết hợp các hàng dữ liệu từ hai hay nhiều bảng dựa trên một cột chung. Cột chung này thường là mối quan hệ giữa Khóa ngoại (Foreign Key) của bảng này và Khóa chính (Primary Key) của bảng kia.

Để thiết lập quy tắc liên kết, chúng ta sử dụng mệnh đề **ON**. Mệnh đề này xác định điều kiện để các hàng từ hai bảng có thể "khớp" với nhau.

2. Các loại phép nối phổ biến

Tùy thuộc vào việc chúng ta muốn xử lý các bản ghi không có sự trùng khớp như thế nào, SQL cung cấp bốn loại phép nối chính:

2.1. INNER JOIN (Nối trong) Đây là phép nối trả về các bản ghi chỉ khi có sự trùng khớp ở cả hai bảng. Về mặt toán học, đây chính là **phép giao (Intersection)** giữa hai tập hợp dữ liệu.

- **Đặc điểm:** Nếu một hàng ở bảng trái không tìm thấy hàng tương ứng ở bảng phải (và ngược lại), hàng đó sẽ bị loại khỏi tập kết quả.

Ví dụ: Truy xuất danh sách sinh viên cùng tên lớp của họ.

```
SELECT s.HoTen, l.TenLop
FROM SinhVien s
INNER JOIN LopHoc l ON s.lop_hoc_id = l.id;
```

- *Lưu ý:* Sinh viên chưa được xếp lớp hoặc lớp chưa có sinh viên sẽ không xuất hiện trong báo cáo này.

2.2. LEFT JOIN (Nối trái) Đây là loại phép nối phổ biến nhất trong thực tế. Nó trả về **tất cả các bản ghi từ bảng bên trái** (bảng đứng trước từ khóa JOIN) và các bản ghi khớp từ bảng bên phải.

- **Đặc điểm:** Nếu bảng bên phải không có bản ghi khớp, hệ thống vẫn giữ lại bản ghi ở bảng bên trái và điền giá trị **NULL** vào các cột của bảng bên phải.

Ví dụ: Liệt kê tất cả sinh viên, bao gồm cả những người chưa được phân vào lớp nào.

```
SELECT s.HoTen, l.TenLop
FROM SinhVien s
LEFT JOIN LopHoc l ON s.lop_hoc_id = l.id;
```

- *Kết quả:* Sinh viên chưa có lớp vẫn xuất hiện trong danh sách với cột **TenLop** hiển thị là **NULL**.

2.3. RIGHT JOIN (Nối phải) Ngược lại với LEFT JOIN, phép nối này trả về **tất cả các bản ghi từ bảng bên phải** và các bản ghi khớp từ bảng bên trái.

- **Đặc điểm:** Nếu bảng bên trái không có bản ghi khớp, các cột của bảng bên trái sẽ hiển thị giá trị **NULL**.

Ví dụ: Liệt kê tất cả các lớp học, bao gồm cả những lớp mới mở chưa có sinh viên theo học.

```
SELECT s.HoTen, l.TenLop
FROM SinhVien s
RIGHT JOIN LopHoc l ON s.lop_hoc_id = l.id;
```

2.4. FULL OUTER JOIN (Nối ngoài đầy đủ) Đây là sự kết hợp của cả LEFT JOIN và RIGHT JOIN. Nó trả về tất cả các bản ghi khi có sự trùng khớp ở một trong hai bảng (Trái hoặc Phải).

- **Đặc điểm:** Đây là **phép hợp (Union)** của hai tập hợp. Mọi bản ghi từ cả hai phía đều được giữ lại; những phần không có sự trùng khớp sẽ được lấp đầy bằng giá trị **NULL**.

Ví dụ: Truy xuất toàn bộ danh sách sinh viên và toàn bộ danh sách lớp học để kiểm tra tổng thể sự phân bố.

```
SELECT s.HoTen, l.TenLop
FROM SinhVien s
FULL OUTER JOIN LopHoc l ON s.lop_hoc_id = l.id;
```

Tổng kết

Phép nối JOIN là công cụ cốt lõi để tái cấu trúc thông tin từ các bảng đã được chuẩn hóa. Việc lựa chọn loại JOIN phù hợp phụ thuộc vào mục tiêu báo cáo:

1. **INNER JOIN:** Khi chỉ cần các dữ liệu có sự liên kết chặt chẽ ở cả hai phía.
2. **LEFT JOIN:** Khi cần ưu tiên hiển thị đầy đủ thực thể chính (thường là bảng bên trái) mà không muốn mất dữ liệu nếu chúng chưa có liên kết.
3. **RIGHT JOIN:** Khi cần ưu tiên hiển thị thực thể tham chiếu (bảng bên phải).
4. **FULL OUTER JOIN:** Khi cần một cái nhìn toàn diện về dữ liệu từ cả hai phía, bao gồm cả các bản ghi khuyết thiếu liên kết.

Việc nắm vững các phép nối này là yếu tố quyết định để lập trình viên có thể xử lý các bài toán truy vấn phức tạp trên nhiều bảng trong các dự án thực tế.